

---

# Debounce & Throttling in React JS

From Beginner → Intermediate → Expert

React

Hooks

Performance

Custom Hooks

DEBOUNCE – "Wait until you stop"

keystroke 

API call 

THROTTLE – "Fire at most once per interval"

scroll 

handler 

300ms 300ms 300ms

# 1 — The Mental Model

Before writing any code, internalize this core distinction:

## DEBOUNCE

## THROTTLE

"Wait until you STOP"

Resets timer

silence.

■ Key insight: Debounce RESETS its timer on each event. Throttle IGNORES events during cooldown.

# 2 — Debounce From Scratch (No Library)

## Custom Hook

```
import { useState, useEffect } from 'react';

// ■ Reusable custom hook
function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    // Timer resets every time 'value' changes
    const timer = setTimeout(() => {
      setDebouncedValue(value); // only updates after user STOPS
    }, delay);

    // ■ Cleanup: cancel previous timer on every re-render
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}
```

## Usage in SearchBar Component

```
function SearchBar() {
  const [inputValue, setInputValue] = useState('');
  const debouncedSearch = useDebounce(inputValue, 600); // waits 600ms

  // Only fires when user STOPS typing for 600ms
  useEffect(() => {
```

```
if (!debouncedSearch) return;

console.log('■ API call with:', debouncedSearch);

// fetchResults(debouncedSearch);

}, [debouncedSearch]); // ← depends on debounced, not raw input

return (

<div>

<input

value={inputValue}

onChange={(e) => setInputValue(e.target.value)}

placeholder='Search...'

/>

<p>Raw input: {inputValue}</p>

<p>Debounced (API uses): {debouncedSearch}</p>

</div>

);

}
```

## 3 — Throttle From Scratch (No Library)

### Custom Hook

```
import { useRef, useCallback } from 'react';

function useThrottle(callback, delay = 300) {

const lastCall = useRef(0); // timestamp of last execution

return useCallback((...args) => {

const now = Date.now();

// Only execute if enough time has passed

if (now - lastCall.current >= delay) {

lastCall.current = now;

callback(...args);

}

}, [callback, delay]);

}
```

```
}
```

## Usage in Scroll Tracker

```
function ScrollTracker() {

  const handleScroll = useThrottle(() => {

    console.log('■ Scroll Y:', window.scrollY);

  }, 300); // fires at most every 300ms

  useEffect(() => {

    window.addEventListener('scroll', handleScroll);

    return () => window.removeEventListener('scroll', handleScroll);

  }, [handleScroll]);

  return <div style={{ height: '2000px' }}>Scroll me!</div>;

}
```

## 4 — Real World: Search + Loading + AbortController

■■ AbortController cancels stale network requests. Without it, a slow earlier request can resolve AFTER a newer one — showing outdated results. This is called a Race Condition.

```
function ProductSearch() {

  const [query, setQuery] = useState('');

  const [results, setResults] = useState([]);

  const [loading, setLoading] = useState(false);

  const [error, setError] = useState(null);

  const debouncedQuery = useDebounce(query, 500);

  useEffect(() => {

    if (!debouncedQuery.trim()) { setResults([]); return; }

    const controller = new AbortController(); // cancel stale requests

    async function fetchProducts() {

      setLoading(true); setError(null);

      try {
```

```
const res = await fetch(`/api/products?search=${debouncedQuery}`,
{ signal: controller.signal });

const data = await res.json();

setResults(data);

} catch (err) {

if (err.name !== 'AbortError') setError('Something went wrong');

} finally { setLoading(false); }

}

fetchProducts();

return () => controller.abort(); // cancel if query changes again

}, [debouncedQuery]);

return (

<div>

<input value={query} onChange={(e) => setQuery(e.target.value)}

placeholder='Search products...' />

{loading && <p>Searching...</p>}

{error && <p style={{ color: 'red' }}>{error}</p>}

<ul>{results.map(p => <li key={p.id}>{p.name}</li>)}</ul>

</div>

);

}
```

## 5 — Real World: Throttled Scroll + Resize

```
function Dashboard() {

const [scrollY, setScrollY] = useState(0);

const [windowSize, setWindowSize] = useState({

w: window.innerWidth, h: window.innerHeight,

});

const [showBackToTop, setShowBackToTop] = useState(false);

// Throttled scroll handler – fires every 200ms max

const handleScroll = useThrottle(() => {
```

```
const y = window.scrollToY;

setScrollY(y);

setShowBackToTop(y > 400);

}, 200);

// Throttled resize handler – fires every 300ms max
const handleResize = useThrottle(() => {
  setWindowSize({ w: window.innerWidth, h: window.innerHeight });
}, 300);

useEffect(() => {
  window.addEventListener('scroll', handleScroll);
  window.addEventListener('resize', handleResize);

  return () => {
    window.removeEventListener('scroll', handleScroll);
    window.removeEventListener('resize', handleResize);
  };
}, [handleScroll, handleResize]);

return (
  <div style={{ height: '3000px', padding: '20px' }}>
    <p>Scroll Y: {scrollY}px</p>
    <p>Window: {windowSize.w} x {windowSize.h}</p>
    {showBackToTop && (
      <button onClick={() => window.scrollTo({ top: 0, behavior: 'smooth' })}
        style={{ position: 'fixed', bottom: 20, right: 20 }}>
        Back to Top
      </button>
    )}
  </div>
);
}
```

## 6 — Using Lodash (Production Standard)

■ Use `useMemo` to create the debounced/throttled function ONCE. Without it, a new function is created on every render — resetting the timer and breaking the behaviour entirely.

## Debounce with Lodash

```
import { useMemo, useEffect } from 'react';
import debounce from 'lodash/debounce';

function SearchWithLodash() {
  // ■ useMemo – created ONCE for component lifetime
  const debouncedFetch = useMemo(
    () => debounce((query) => {
      console.log('Fetching:', query);
    }, 500),
    []
  );

  // ■ Cancel pending calls on unmount – prevents memory leaks
  useEffect(() => {
    return () => debouncedFetch.cancel();
  }, [debouncedFetch]);

  return (
    <input onChange={(e) => debouncedFetch(e.target.value)}
      placeholder='Search...' />
  );
}
```

## Throttle with Lodash

```
import throttle from 'lodash/throttle';

function ScrollWithLodash() {
  const throttledLog = useMemo(
    () => throttle(() => {
      console.log('Scroll:', window.scrollY);
    }, 300),
    []
  );
```

```
);

useEffect(() => {
  window.addEventListener('scroll', throttledLog);
  return () => {
    throttledLog.cancel(); // flush any pending call
    window.removeEventListener('scroll', throttledLog);
  };
}, [throttledLog]);

return <div style={{ height: '2000px' }}>Scroll!</div>;
}
```

## 7 — Debounce on Button Click

Useful for preventing accidental double-click form submissions (e.g. payment buttons).

```
import { useRef } from 'react';

function PaymentButton({ onPay }) {
  const timerRef = useRef(null);

  const handleClick = () => {
    clearTimeout(timerRef.current); // cancel any pending call

    timerRef.current = setTimeout(() => {
      onPay(); // fires only if NOT clicked again within 800ms
    }, 800);
  };

  return (
    <button onClick={handleClick}>
      Confirm Payment
    </button>
  );
}
```

## 8 — When to Use Which

## Use Cases Comparison

| Use Case          | Debounce ■                     | Throttle ■                   |
|-------------------|--------------------------------|------------------------------|
| Search input      | ■ Wait for user to stop typing |                              |
| Form validation   | ■ Validate after field blur    |                              |
| Autosave          | ■ Save after user pauses       |                              |
| Scroll events     |                                | ■ Track at steady pace       |
| Window resize     |                                | ■ Update layout in real-time |
| Mouse move / drag |                                | ■ Smooth position updates    |
| Analytics events  |                                | ■ Max N events per second    |
| URL/query sync    | ■ Update after typing stops    |                              |

## 9 — Critical Gotchas

### ■ WRONG — New debounced function on every render

```
// ■ WRONG – debounce() is called on EVERY render

// The timer resets each render – debounce NEVER fires

function Bad() {

  const handleChange = debounce((val) => fetchAPI(val), 500);

  return <input onChange={(e) => handleChange(e.target.value)} />;

}
```

### ■ CORRECT — Stable reference with useMemo

```
// ■ CORRECT – created ONCE, stable across renders

function Good() {

  const handleChange = useMemo(

    () => debounce((val) => fetchAPI(val), 500),

    []

  );

  return <input onChange={(e) => handleChange(e.target.value)} />;

}
```

### ■ Memory Leak — No cleanup on unmount

```
// ■ Pending debounced call fires AFTER component unmounts
```

```
useEffect(() => {  
  // No cleanup – memory leak!  
}, []);  
  
// ■ Always cancel on unmount  
useEffect(() => {  
  return () => debouncedFn.cancel(); // lodash  
  // or: return () => clearTimeout(timerRef.current); // manual  
}, []);
```

## 10 — Summary Comparison Table

| Property               | Debounce                     | Throttle                |
|------------------------|------------------------------|-------------------------|
| Fires when             | After silence / user stops   | At regular intervals    |
| Resets timer on event? | ■ Yes                        | ■ No                    |
| Best for               | Search, validation, autosave | Scroll, resize, drag    |
| Risk if skipped        | Too many API calls           | Janky/missed UI updates |
| Lodash method          | _.debounce(fn, ms)           | _.throttle(fn, ms)      |
| Must cancel on unmount | ■ Always                     | ■ Always                |
| Common delay range     | 300ms – 800ms                | 100ms – 500ms           |

■ Golden Rule: Always use `useMemo` (or `useRef`) to keep a stable function reference. Always cancel (cleanup) on unmount. Use `AbortController` for fetch-based debounce.